

Revisão



VARIÁVEIS,
TIPOS,
ENTRADA E
SAÍDA

```
1 nome = input("Digite o nome do cliente: ").strip()
2 idade = int(input("Digite a idade: "))
3 saldo = float(input("Digite o saldo inicial: "))
4
5 print("Cliente:", nome)
6 print("Idade:", idade)
7 print("Saldo:", saldo)
```

OPERADORES
RELACIONAIS E
LÓGICOS + IF /
ELIF / ELSE

```
1 valor = float(input("Valor da compra: "))
2
3 if valor >= 1000:
4     desconto = 0.10
5 elif valor >= 500:
6     desconto = 0.05
7 else:
8     desconto = 0
9
10 total = valor - (valor * desconto)
11 print("Total com desconto:", total)
```

MANIPULAÇÃO
DE STRINGS
(strip, lower,
upper, len, in,
slicing)

```
1 email = input("Digite o e-mail: ").strip().lower()
2
3 if "@" in email and "." in email and len(email) >= 6:
4     dominio = email[email.find("@") + 1:]
5     print("Email válido. Domínio:", dominio)
6 else:
7     print("Email inválido.")
```

```
1 # Solicita ao usuário que digite um e-mail pelo teclado
2 email = input("Digite o e-mail: ")
3
4 # Remove espaços em branco do início e do fim do texto digitado e converte todos os caracteres para letras
  minúsculas, garantindo padronização para validação e comparação.
5 email = email.strip().lower()
6
7 # Verifica três condições ao mesmo tempo:
8 # 1) Se o caractere "@" existe dentro do e-mail
9 # 2) Se o caractere "." existe dentro do e-mail
10 # 3) Se o tamanho total do texto é maior ou igual a 6 caracteres
11 # Todas as condições precisam ser verdadeiras para o e-mail ser considerado válido
12 if "@" in email and "." in email and len(email) >= 6:
13
14     # Localiza a posição do caractere "@" - Soma 1 para iniciar o recorte após o "@" - Usa slicing para extrair
      apenas o domínio do e-mail
15     dominio = email[email.find("@") + 1:]
16
17     # Exibe uma mensagem informando que o e-mail é válido e mostra o domínio extraído
18     print("Email válido. Domínio:", dominio)
19
20 else:
21     # Executado caso qualquer uma das condições de validação falhe - Indica que o e-mail não atende aos critérios
      mínimos
22     print("Email inválido.")
```

WHILE +
VALIDAÇÃO +
CONTROLE DE
FLUXO

```
1 senha = ""
2
3 while senha != "1234":
4     senha = input("Digite a senha: ").strip()
5     if senha != "1234":
6         print("Senha incorreta.")
7
8 print("Acesso liberado.")
```



WHILE TRUE +
BREAK +
CONTINUE

```
1 while True:
2     idade = int(input("Digite a idade (>=18): "))
3
4     if idade < 18:
5         print("Idade inválida.")
6         continue
7
8     print("Idade aceita.")
9     break
```

WHILE TRUE + BREAK + CONTINUE

```
1 # Inicia um laço de repetição infinito. O valor True faz com que o while nunca pare sozinho. A saída do
  laço só acontece quando um break for executado.
2 while True:
3
4     # Solicita que a idade seja digitada pelo usuário. O input retorna texto, por isso é convertido para
    inteiro com int()
5     idade = int(input("Digite a idade (>=18): "))
6
7     # Verifica se a idade informada é menor que 18
8     if idade < 18:
9
10        # Exibe uma mensagem informando que a idade não atende ao critério mínimo
11        print("Idade inválida.")
12
13        # O continue interrompe apenas a iteração atual do while e faz o fluxo voltar para o início do
        laço, solicitando a idade novamente
14        continue
15
16    # Executado somente quando a idade é maior ou igual a 18. Indica que a condição foi atendida
17    print("Idade aceita.")
18
19    # O break encerra definitivamente o laço while fazendo o programa sair da repetição infinita
20    break
```


FOR SIMPLER
(PERCORRENDO
DADOS)

```
1 usuarios = ["ana", "bruno", "carla", "diego"]  
2  
3 for usuario in usuarios:  
4     print(usuario.title())
```

FOR + RANGE
(CONTROLE DE
PROCESSO)

```
1 for i in range(1, 11):  
2     print("Processando item", i)
```

FOR + RANGE
COM PASSOS
(LOTE /
PAGINAÇÃO)

```
1 registros = list(range(1, 51))  
2  
3 for i in range(0, len(registros), 10):  
4     print("Lote:", registros[i:i+10])
```

FOR + RANGE
COM PASSOS
(LOTE /
PAGINAÇÃO)

```
1 # Cria uma lista chamada registros contendo números de 1 até 50. A função range(1,
   # 51) gera números de 1 a 50. O list() converte essa sequência em uma lista real
2 registros = list(range(1, 51))
3
4 # Inicia um laço for que controla os índices da lista range(0, len(registros), 10)
   # significa:
5 # - começar no índice 0
6 # - parar no tamanho total da lista (50)
7 # - avançar de 10 em 10 posições
8 for i in range(0, len(registros), 10):
9
10     # Usa slicing para extrair um "lote" de 10 registros por vez, registros[i:i+10]
       # pega os elementos do índice i até i+9. Isso simula a divisão de dados em blocos
       # ou páginas
11     print("Lote:", registros[i:i+10])
```

FOR +
CONTINUE +
BREAK
(CONTROLE DE
FLUXO)

```
1 lotes = [10, 0, 25, -1, 18, 30]
2
3 for qtd in lotes:
4     if qtd == 0:
5         continue
6     if qtd < 0:
7         break
8     print("Processando lote:", qtd)
```

FOR + CONTINUE + BREAK (CONTROLE DE FLUXO)

```
1 # Cria uma lista chamada lotes contendo quantidades a serem processadas. Cada valor representa a quantidade  
  de itens em um lote  
2 lotes = [10, 0, 25, -1, 18, 30]  
3  
4 # Inicia um laço for que percorre cada valor da lista lotes  
5 for qtd in lotes:  
6  
7     # Verifica se a quantidade do lote é igual a zero. Um lote com quantidade zero não precisa ser  
    processado  
8     if qtd == 0:  
9  
10        # O continue faz o laço pular imediatamente para o próximo item. Ignora completamente este lote e  
        não executa o restante do código  
11        continue  
12  
13    # Verifica se a quantidade é negativa. Valores negativos indicam erro crítico nos dados  
14    if qtd < 0:  
15  
16        # O break encerra o laço for imediatamente. Nenhum outro lote será analisado após esse ponto  
17        break  
18  
19    # Executado somente se a quantidade for maior que zero. Indica que o lote é válido e está sendo  
    processado  
20    print("Processando lote:", qtd)
```

FOR +
CONTINUE +
TRATAMENTO
DE ERROS
(CONTROLE DE
FLUXO)

```
1  lotes = [10, 0, 25, -1, 18, 30]
2
3  for qtd in lotes:
4      # Ignora lote zerado
5      if qtd == 0:
6          print("Lote zerado ignorado.")
7          continue
8
9      # Trata erro crítico sem encerrar o sistema
10     if qtd < 0:
11         print("Erro encontrado no lote:", qtd)
12         continue
13
14     # Processa apenas lotes válidos
15     print("Processando lote:", qtd)
```



O que mudou tecnicamente

- Removido break → evita encerrar o laço
- Usado continue → permite seguir analisando os próximos dados
- Sistema fica mais robusto
- Ideal para:
 - processamento em lote
 - auditorias
 - integração com banco de dados
 - sistemas administrativos e logísticos

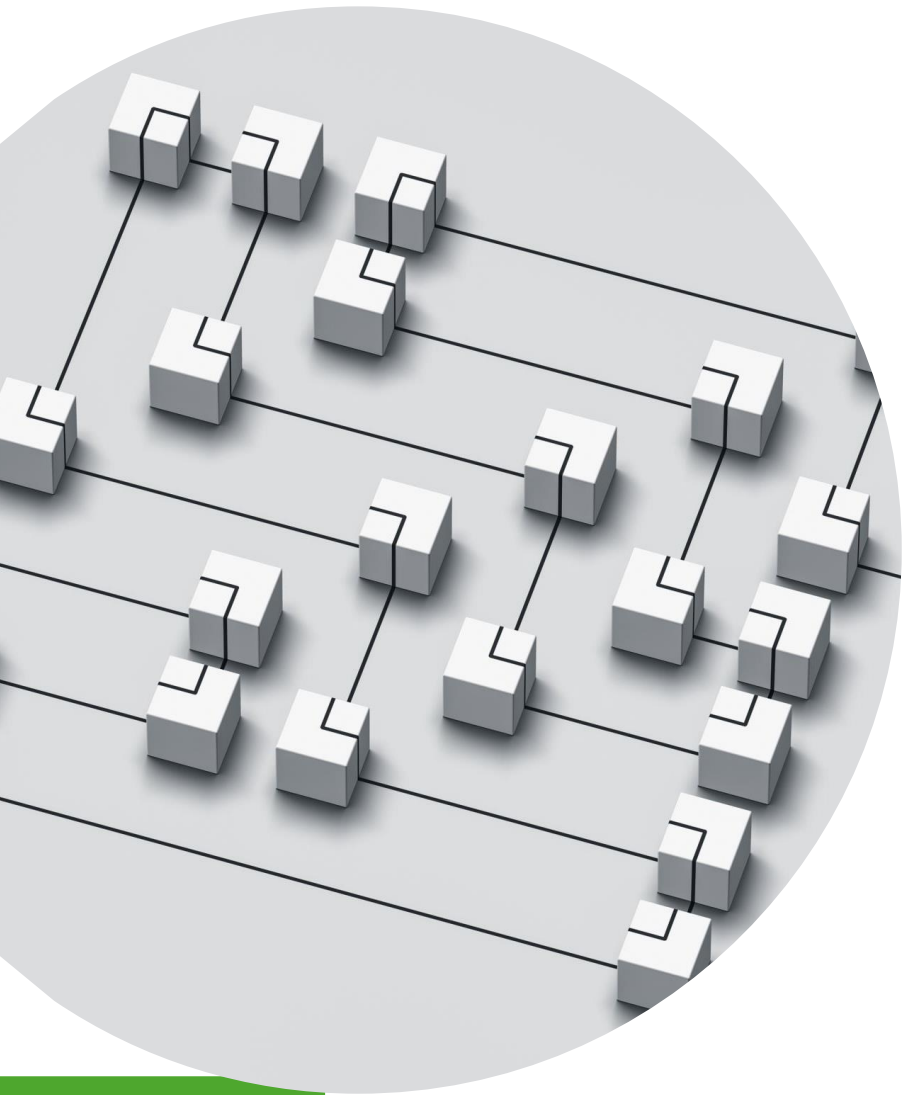
Regra profissional importante:

- continue → quando o erro não impede o sistema de continuar
 - break → somente quando o erro invalida todo o processo
- 

FOR +
CONTINUE +
TRATAMENTO
DE ERROS
(CONTROLE DE
FLUXO)

```
1  lotes = [10, 0, 25, -1, 18, 30]
2
3  for qtd in lotes:
4      # Ignora lote zerado
5      if qtd == 0:
6          print("Lote zerado ignorado.")
7          continue
8
9      # Trata erro crítico sem encerrar o sistema
10     if qtd < 0:
11         print("Erro encontrado no lote:", qtd)
12         continue
13
14     # Processa apenas lotes válidos
15     print("Processando lote:", qtd)
```

Exercício

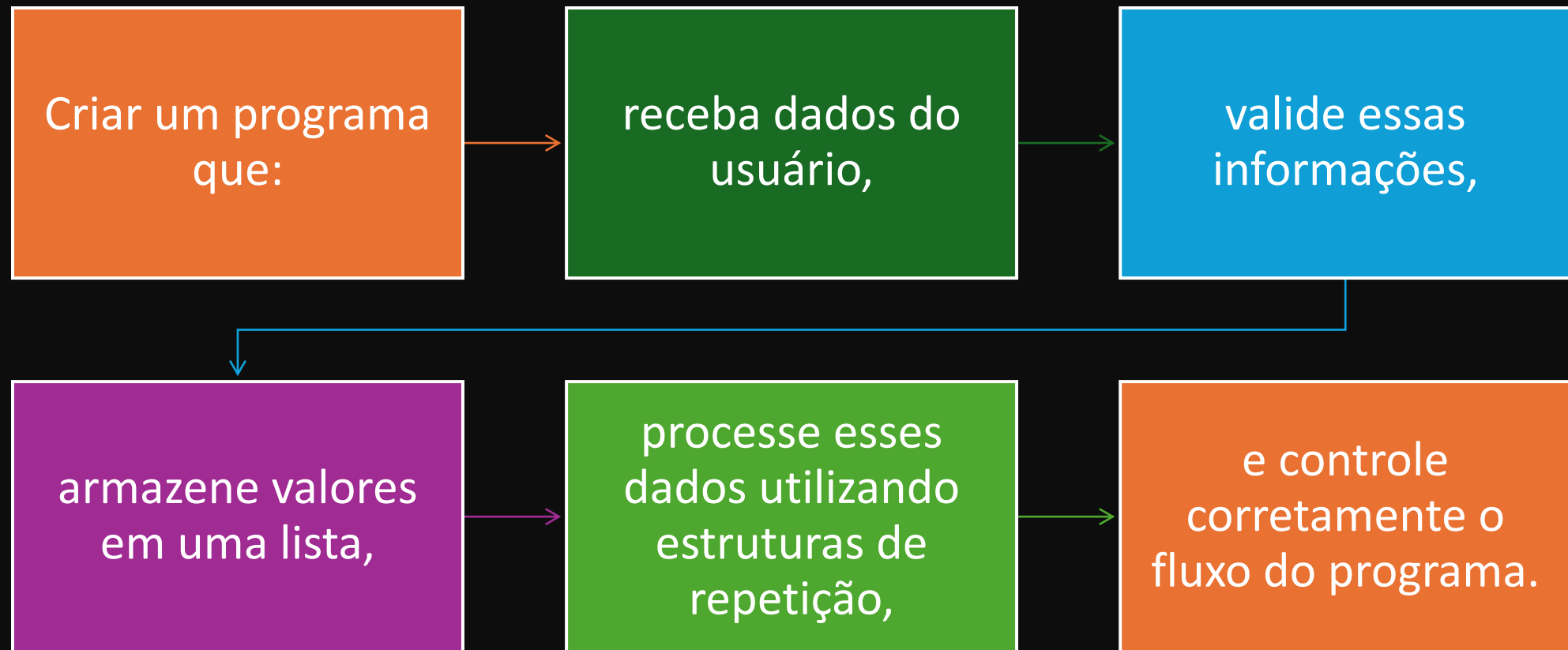


Esse tipo de lógica é comum em:

- sistemas administrativos internos;
- controle de produção;
- triagem de dados antes de envio ao banco de dados;
- validação de entradas humanas;
- rotinas de auditoria e conferência.

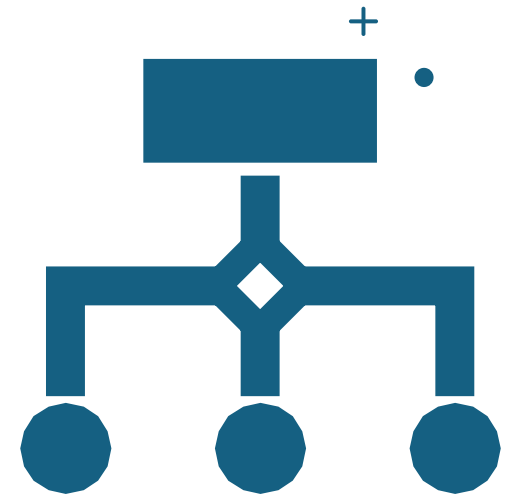
Ele demonstra claramente que lógica de programação e técnicas de programação não são conceitos abstratos, mas ferramentas práticas para resolver problemas reais de empresa.

Objetivo do projeto



Regras do projeto (passo a passo obrigatório)

1. O sistema deve iniciar com uma lista vazia chamada registros.
2. O sistema deve usar um while para permitir que o usuário cadastre vários valores.
3. A cada repetição do while, o usuário deve digitar um valor inteiro:
 - se o valor digitado for 0, o sistema deve ignorar o valor usando continue;
 - se o valor digitado for negativo, o sistema deve encerrar o cadastro usando break;
 - se o valor for positivo, ele deve ser adicionado à lista registros.
4. Após o encerramento do cadastro, o sistema deve usar um for para percorrer a lista registros.
5. Durante o for:
 - se o valor for menor que 10, o sistema deve ignorar esse valor usando continue;
 - se o valor for maior ou igual a 100, o sistema deve encerrar o processamento usando break;
 - caso contrário, o sistema deve exibir a mensagem "Processando registro: X".
6. Ao final, o sistema deve exibir:
 - a lista completa de registros cadastrados;
 - uma mensagem indicando que o processamento foi finalizado.



```
1 registros = []
2
3 # Etapa 1 – Cadastro dos valores
4 while True:
5     valor = int(input("Digite um valor inteiro (0 ignora, negativo encerra): "))
6
7     if valor == 0:
8         continue
9
10    if valor < 0:
11        break
12
13    registros.append(valor)
14
15 # Etapa 2 – Processamento dos registros
16 for item in registros:
17     if item < 10:
18         continue
19
20     if item >= 100:
21         break
22
23     print("Processando registro:", item)
24
25 # Etapa 3 – Finalização
26 print("Registros cadastrados:", registros)
27 print("Processamento finalizado.")
```

O que é o comando append()

Em Python, listas servem para guardar vários valores juntos. O comando:

```
registros.append(valor)
```

significa, literalmente: “Adicionar o valor no final da lista registros.”

O que acontece na prática

Imagine que a lista começa vazia:

- `registros = []`

Quando o código executa:

- `registros.append(10)`

A lista passa a ser:

- `[10]`

Se depois executar:

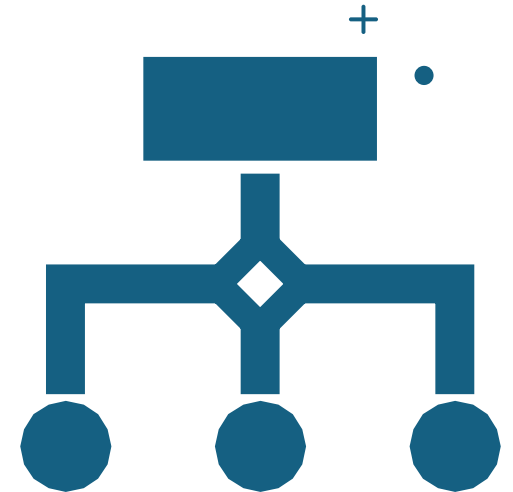
- `registros.append(25)`

A lista vira:

- `[10, 25]`

Ou seja:

- `append()` não substitui valores
- `append()` não altera os anteriores
- Ele apenas acrescenta um novo item ao final da lista



O que é o comando append()

Por que esse comando foi usado no projeto

No projeto enviado, o sistema precisava:

- receber vários valores digitados pelo usuário;
- guardar esses valores para processar depois com for.

Sem append(), não existe como armazenar múltiplos dados dinamicamente em uma lista durante a execução.

O append() é o comportamento mínimo necessário para:

- cadastro,
- acúmulo de dados,
- processamento posterior.

Ligação direta com lógica

Pensando apenas em lógica, o append() faz isso: “Toda vez que o usuário digitar um valor válido, guarde ele para usar depois.”

Isso não muda o raciocínio, só executa a ação.

Observação:

O comando append() é usado para adicionar um valor ao final de uma lista.

Sempre que o sistema recebe um valor válido, ele é armazenado na lista para ser processado depois.

